Introduction to Agent Skills

SKILL.md · Frontmatter · Matching · Sharing

mostly Opus 4.6

Previously, on Claude Code in Action...

Tool Use

LLMs interact with the world through tool calls — read files, run commands, edit code.

MCP Servers

Model Context Protocol extends Claude with new tools. Deferred loading keeps context lean.

CLAUDE.md & Context

Persistent context file included in every request. `@` mentions for file focus. `/compact` vs. `/clear`.

Hooks & SDK

PreToolUse/PostToolUse hooks for validation and formatting. Claude Agent SDK for programmatic control.

Permissions

Three tiers (read/bash/edit). Five modes from `default` to `bypass`. Pre-approve in `settings.json`.

What Are Skills?

Skills = reusable markdown instructions that Claude automatically applies to the right tasks at the right time.

Key properties

- Each skill lives in a `SKILL.md` file with **name** and **description** in frontmatter
- Claude matches your request against skill descriptions — activates automatically
- Load *on demand* (unlike `CLAUDE.md` which loads every time)

Rule of thumb: if you find yourself explaining the same thing to Claude repeatedly, that's a skill waiting to be written.

Bundled skills: `/batch`, `/simplify`, `/debug`, `/claude-api` ship with Claude Code.

Skills vs. Other Features

Feature	When loaded	Activation	Scope
`CLAUDE.md`	Every conversation	Always	Project-wide
Slash commands	On invocation	Explicit (`/cmd`)	Project/personal
Skills	On demand	**Automatic** (semantic)	Project/personal
Hooks	On tool use	Trigger-based	Settings file
MCP servers	On tool use	Tool-based	Config file
Subagents	On delegation	Automatic	Agent definition

Skills activate when Claude *recognizes the situation* — no explicit invocation needed.

Where Skills Live

Personal Skills

- Location: `~/.claude/skills/`
- Follow you across *all* projects
- Your personal coding preferences

Project Skills

- Location: `.claude/skills/`
- Shared with anyone who clones the repo
- Team conventions & workflows

Priority hierarchy (name conflicts):

Enterprise → Personal → Project → Plugins

Creating Your First Skill

Directory structure:

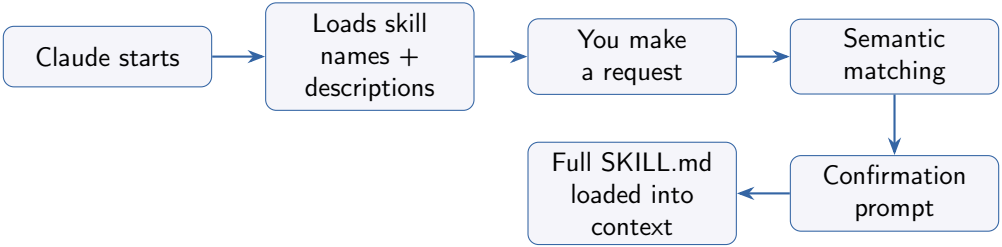
```
~/.claude/skills/
  pr-description/
    SKILL.md
```

SKILL.md with frontmatter:

```
---
name: pr-description
description: >
  Generate a pull request description from the
  current branch's changes. Summarize what changed,
  why, and any testing notes.
---

When asked to create a PR description:
1. Run `git diff main...HEAD` to see all changes
2. Summarize the purpose of the changes
3. List key files modified
4. Note any breaking changes or migration steps
5. Suggest reviewers based on file ownership
```

How Skill Matching Works



Important

Only names and descriptions are loaded at startup. The full skill body is loaded *only after confirmation* — keeps context window lean.

Configuration & Multi-File Skills

Frontmatter options:

```
---
name: deploy-checker
description: Validate deployment readiness
allowed-tools:
  - Bash
  - Read
model: sonnet
context: fork # run in isolated subagent
user-invocable: true # show in / menu
---
```

Progressive disclosure — multi-file skills:

```
.claude/skills/deploy-checker/
  SKILL.md # main entry point
  scripts/ # shell scripts, Python, etc.
  templates/ # report templates
```

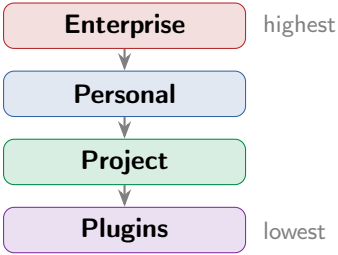
Skills can reference supporting files, scripts, and templates. Dynamic context injection available at load time.

Sharing Skills & Priority Hierarchy

How to share:

- **Git** — commit `.claude/skills/` to repo; team gets skills on clone
- **Plugins** — installable packages (growing directory), open standard (`agentskills.io`)
- **Enterprise** — org-wide deployment, enforce standards at scale
- **Composition** — skills can invoke subagents for powerful workflows

Priority (name conflicts):



Troubleshooting Skills

Common Issues

1. **Skill doesn't trigger** — description too vague; make it specific and action-oriented
2. **Wrong skill triggers** — descriptions overlap; differentiate with distinct verbs/contexts
3. **Skill not loading** — check file path, frontmatter YAML syntax, restart Claude
4. **Priority conflict** — remember: Enterprise > Personal > Project > Plugins
5. **Runtime errors** — test skill instructions manually first

Tip: Verify skill loading

Restart Claude Code and confirm your skill appears in the skill list. If it doesn't, check the file path and YAML frontmatter syntax.

Summary

Key Takeaways

- Skills = **reusable instructions** that activate automatically via semantic matching
- Frontmatter: `name`, `description`, `allowed-tools`, `model`, `context`, `user-invocable`
- Personal (`~/.claude/skills/`) vs. Project (`.claude/skills/`)
- Write specific descriptions — they are the matching key
- Share via git, plugins, or enterprise managed settings
- Multi-file skills with progressive disclosure and dynamic context injection

Questions?